

Lesson 1 — What Is A Game, Really?

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

The three jobs, every frame

INPUT	what is the player asking for?
UPDATE	change the numbers
RENDER	erase, then draw

Then repeat. Forever. That is the **game loop**.

The skeleton

```
function update() {
  // change numbers only
}

function render() {
  // erase first
  ctx.fillStyle = "#222";
  ctx.fillRect(0, 0, 600, 400);
  // then draw
  ctx.fillStyle = "#4a9eff";
  ctx.fillRect(x, y, 40, 40);
}

function frame() {
  update();
  render();
  requestAnimationFrame(frame);
}

frame();
```

Getting a canvas

```
const canvas =
  document.getElementById("game");
const ctx = canvas.getContext("2d");
```

Drawing

```
ctx.fillStyle = "#4a9eff";    // colour FIRST
ctx.fillRect(x, y, w, h);     // then draw
```

(0, 0) is the **top-left** corner.

Words

Frame — one trip round the loop, one picture. **State** — everything the game remembers. **fps** — frames per second, usually about 60. **ctx** — the drawing context; works like a pen.

The three rules worth memorising

1. Update changes numbers. Render draws. Never mix them.
2. The canvas never cleans itself. **Erase, then draw.**
3. Set the colour **before** you draw, not after.

Section A — Recall

A1. Name the three jobs a game does every frame, in the correct order.

[3 marks]

A2. What does the word **state** mean when we are talking about a game?

[2 marks]

A3. A game runs at 60 frames per second. How many pictures does a player look at during a four-minute level? Show your working.

[2 marks]

A4. On a canvas, where is the point `(0, 0)`?

[2 marks]

A5. Explain the flipbook comparison in your own words. Then say one way a game is *not* like a flipbook.

[3 marks]

Section B — Predict the output

Write your prediction down before you run anything. Being wrong on paper is where the learning happens — nobody is marking whether you guessed right.

B1. What does this draw? Be precise about the colour.

[3 marks]

```
ctx.fillRect(10, 10, 100, 50);  
ctx.fillStyle = "red";
```

B2. The square starts at `squareX = 50`. What is `squareX` after five frames, and how far has the square moved on screen?

[3 marks]

```
function update() {  
    squareX = squareX + 2;  
}
```

B3. What appears on screen here, and why is it not what the author intended?

[4 marks]

```
function frame() {  
  update();  
  render();  
  // requestAnimationFrame(frame);  
}  
frame();
```

B4. This swaps two lines. Describe exactly what the player would see, and explain why it is nearly but not quite the same as the correct version.

[4 marks]

```
function frame() {  
  render();  
  update();  
  requestAnimationFrame(frame);  
}
```

Section C — Find and fix the bug

C1. A student reports: “my square leaves a long streak behind it, like a paint smear.” Here is their `render`. What is missing, and why does that cause a streak rather than, say, nothing at all?

[4 marks]

```
function render() {  
  ctx.fillStyle = "#4a9eff";  
  ctx.fillRect(squareX, squareY, 40, 40);  
}
```

C2. Nothing appears at all, and the browser console says `Cannot read properties of null (reading 'getContext')`. What does that message mean in plain English, and what is wrong with this page?

[4 marks]

```
<script>  
  const canvas = document.getElementById("game");  
  const ctx = canvas.getContext("2d");  
</script>  
  
<canvas id="game" width="600" height="400"></canvas>
```

C3. This square never moves, even though `update` looks correct. Why not?

[3 marks]

```
let squareX = 50;

function update() {
  let squareX = squareX + 2;
}
```

Section D — Build it

Work through these in order. Tick each checkpoint when it works. Reaching checkpoint 4 is a full pass — 5 and 6 are there for people who are flying.

- ☐ **Checkpoint 1.** Make a new file called `mygame.html`. Get a canvas on the page, 600×400, with a dark background. Nothing else.
- ☐ **Checkpoint 2.** Draw a coloured square at a position held in two variables, `squareX` and `squareY`. Prove the variables are really being used by changing their values and reloading.
- ☐ **Checkpoint 3.** Add `update()`, `render()` and `frame()`, and get the square moving to the right on its own. Remember the erase.
- ☐ **Checkpoint 4.** When the square goes off the right edge, make it reappear on the left. *Careful:* if you reset it to `0` it will visibly pop. What value gives a smooth wrap-around?
- ☐ **Checkpoint 5.** Add a second square of a different colour, at a different height, moving at a different speed. How many new variables did you need?
- ☐ **Checkpoint 6.** Make one of the squares move diagonally — down as well as across — and bounce off the bottom edge.

IF YOU GET STUCK

Open the browser console with `F12` and read the red text. It tells you the file and the

line number. That one habit will save you more time this term than anything else in this handout.

And if nothing at all happens: are you editing the same file you are looking at in the browser?

Did you save it? Did you reload the page?

Section E — Design challenge

NO RIGHT ANSWER

This is marked on your reasoning, not on reaching any particular conclusion. Say what your idea costs as well as what it gains.

A racing game runs its loop 60 times a second, non-stop. A chess program sits and waits, perhaps for several minutes, until someone moves a piece. Both are games.

E1. List three games that need a continuous loop and three that could just wait for input.

E2. In one sentence, what is the real difference between the two groups?

E3. A chess program *could* be written with a 60-per-second loop that mostly does nothing at all. What would that cost? Now think about a phone running on battery — does that change your answer?

E4. Going the other way: suppose you wrote a racing game that only updated when a key was pressed. Describe exactly what the player would experience.

Stretch goals

Never required.

1. **Trail.** Erase with a see-through black (`ctx.fillStyle = "rgba(0,0,0,0.08)"`) instead of a solid one. Explain why this gives a fading trail.
2. **Frame counter.** Count frames, draw the number with `ctx.fillText(count, 10, 30)`, and leave it running for exactly ten seconds. What number do you get? What does that tell you about your computer?
3. **Circle.** Make a square travel in a circle. You will need `Math.sin` and `Math.cos`, which you have not been taught. Experiment: what does `Math.sin(t)` give you as `t` goes up?
4. **The hard one.** Two students' squares move at visibly different speeds on different laptops, with identical code. Why might that be? (This is lesson 2.)